
Android APK — Release Candidate

A finding-by-finding response to the pre-final-testing findings for the APK, frontend and backend: what was actually inside the shipped artifact, the eight defects that were not true of the code and are now fixed, and the four things we need from you.

Aajoo Homes — Android APK · Release Candidate

Prepared 6 September 2026 by the ZypheX Tech development team

Verified against the live development backend and database

1. Response summary

This answers **AAJOO HOMES — Developer Findings Before Final Testing (Android APK + Frontend + Backend)**, ID by ID, in the order the document raises them.

Its opening request is that the update should not simply state "fixed" — it should say what changed and provide evidence. So the two release blockers about the APK are answered the only way they can be: **we took the shipped build apart and read the strings out of it**. Both were true. Both are fixed, and there is now a tool in the repository that reads a built APK back and refuses it if either comes back.

Eight findings were not true of the code when we checked. Two of them were live on production and neither was visible from any screen: an admin endpoint that took no authentication, and socket connections that were anonymous while both clients believed they were authenticating.

Response at a glance	
Findings answered	10 P0 + 15 P1 + 18 frontend + 18 backend
Already implemented, verified	45
Fixed during this review	8
Needs something from you before we can finish it	4 — see section 8
Backend	<code>6ab9cbb</code>
Frontend (website)	<code>b8f3818</code>
Android	<code>1.0.0 (30)</code>
Backend test suite	62/62 files pass

On evidence. The APK findings were checked against the compiled artifact — `classes*.dex`, `lib/*/libapp.so` and the manifest — not against the source. The backend findings were checked against the live API where an unauthenticated request could reach them, and against the code where they could not. Every row says which.

2. What was actually inside the shipped APK

`app-release.apk`, build 29 — the artifact you have. Scanned by `tool/verify_release_apk.py`, which is committed alongside this response.

Looked for	Found	Finding
Development API host	<code>aaजाodev.onrender.com</code> — present	P0-01, FE-01 confirmed
Razorpay test key	<code>rzp_test_XUTODhUdMAshi6</code> — present	P0-02, FE-02 confirmed

Looked for	Found	Finding
Razorpay live key	absent	—
Emulator loopback 10.0.2.2	absent	FE-03 clean
Windows developer paths (C:\Users\...)	absent	FE-03 clean
Unix developer paths	absent — the /home/... strings are Dart package paths (rent_home/ui/screens_renter/home/...), not machine paths	FE-03 clean
Plain-http endpoints	absent — the only http:// strings are XML namespace identifiers (schemas.android.com, ns.adobe.com) and an Android library constant, none of them fetched	FE-18 clean
Cleartext traffic permitted in the manifest	no usesCleartextTraffic, no network-security-config override	FE-18 clean
Certificate-validation bypass	no badCertificateCallback, no HttpOverrides anywhere in the app	FE-18 clean
Release signing	configured, with the keystore supplied from a properties file rather than committed	Section 7 clean

So two of the eleven checks failed, and they are the two the document leads with.

3. Section 2 — Release-blocking P0

ID	Finding	Status	What changed, and how we know
P0-01	Production APK configured with a development backend	Fixed	The default endpoint is now compiled out of release builds: <code>kReleaseMode</code> is a compile-time constant, so in a release build the fallback folds to an empty string and the host is not in the binary at all. A release APK therefore has no endpoint unless one is named at build time, and the app refuses to start without one — it shows a build-configuration screen naming the missing flag, because an app silently talking to nowhere looks exactly like a broken network. Debug builds are unaffected. One thing needs your decision — see section 8.
P0-02	Razorpay TEST configuration present in the APK	Fixed — proved by building one	Same mechanism: the bundled test key is compiled out of release builds. Evidence: we built a release APK passing only the endpoint and no payment key, and scanned it under production rules — no <code>rzp_test_</code> string anywhere in it. Checkout already refused to open in a release build carrying a test key unless someone set <code>ALLOW_TEST_PAYMENTS</code> deliberately, but the finding is about the string being in the artifact, and it was. A shippable production APK still needs your <code>rzp_live_</code> key; build 30 is a QA candidate that carries the sandbox key because it was passed explicitly, and the verifier says so in its output.

ID	Finding	Status	What changed, and how we know
P0-03	Hardcoded secrets must not exist	Done	Configuration reads from the environment; no credential literal remains in <code>config/</code> . Readiness is reported at <code>/health/env</code> — publicly as ready/not-ready, in detail only with the health token. Startup does not currently <i>abort</i> on a missing secret: it logs the missing list and refuses readiness, because killing a running service over a variable that is only needed by one integration is the worse failure. If you would rather it exit, say so and we will make it exit.
P0-04	Admin property endpoints require complete authorization	Fixed — this was live	<code>/admin/properties/load</code> took no authentication at all and answered on production; verified 6 September, when an unauthenticated POST reached the handler and failed on the missing file rather than on authorization. It inserts a property row per spreadsheet line under host id 1. (It could not actually import anything — it was wired to the <i>image</i> uploader, whose filter rejects every spreadsheet — so the endpoint was both unguarded and broken. Both halves are fixed.) <code>/admin/properties/search</code> was declared twice , once guarded and once not, and the guarded one won only because <code>adminProperty.routes.js</code> sorts before <code>property.routes.js</code> in the loader. The duplicate and its handler are gone. A sweep test now fails the build if any <code>/admin</code> route appears without a guard. Verified live after deploy: both answer 401.
P0-05	Sensitive request data must not be logged	Done	Request bodies are redacted centrally, matched on the key name case- and separator-insensitively, so <code>cred_user_password</code> is caught as readily as <code>password</code> . Password, OTP, PIN, token, authorization, JWT, secret, API key, signature and the KYC and bank fields are all on the list.
P0-06	Logout must invalidate server-side	Done	Logout used to write a log row and return success while the token kept working — deleting the client's copy <i>was</i> the logout. There is a durable revocation store now: every session carries an id, logout revokes it, and every authenticated request checks the denylist. Covered by <code>tests/sessionRevocation.test.js</code> .
P0-07	Host authorization: role, status, ownership	Done	Every listing and property mutation is scoped by <code>property_host_id</code> from the token; publishing and payouts both refuse a host whose identity check is not current; a listing published for an unverified host is held inactive until they verify.
P0-08	Private uploaded documents must not be public	Done	Identity, ownership and KYC documents are stored as authenticated assets and served only through short-lived signed URLs. The <code>uploads/</code> directory is no longer served as static files — it used to be, and an invoice names the guest, the property, the dates and the amount.
P0-09	Payment verification atomic and idempotent	Done	The payment row is loaded <code>FOR UPDATE</code> , scoped to the caller. A replay carries a <i>valid</i> signature, so signature checking alone cannot reject it; the lock makes the second arrival wait, and it then sees "already paid" and returns success with no booking update, no host earning, no notification and no ledger row. Before this each replay created a duplicate earning — and earnings feed the payout balance.
P0-10	Guest critical flows connected to real services	Done	Searched the whole Flutter source for scaffold behaviour: no mock data, no stub authentication, no demo property fixtures, no stock image URLs . Two <code>TODOS</code> comments remain in the entire app, neither in a customer-critical path.

4. Section 3 — High priority P1

ID	Requirement	Status	Detail
P1-01	60-minute access token + refresh behaviour	Proposed, not done — see section 8	Tokens are 30 days with server-side revocation. Cutting the access token to 60 minutes without a refresh token would sign every user out hourly, so this is one coordinated change across backend, website and app. We would rather schedule it with its own test cycle than fold it into a release candidate we cannot re-test end to end. The part that limits an abandoned session — P1-02 — is done.
P1-02	30-minute inactivity timeout	Done	Independent of JWT expiry, as the rule asks. Any authenticated request resets the clock, so an active session is never cut off; a session idle past 30 minutes is revoked through the same denylist logout uses, which makes it durable. Applied in the one function every authenticated request passes through, so no auth path can miss it. A session unknown after a restart starts the clock rather than being signed out — the alternative is logging everyone out on every deploy.
P1-03	OTP: 3 attempts, 5-minute expiry, 5 requests/15 min	Done — all three	Attempts were 5 and are now 3 ; the window was 10 minutes and is now 5 ; the request budget was already 5 per 15 minutes. <i>This corrects what we told you in the website findings response, which stated the attempt limit as 5 — that document was written before this one supplied the approved numbers.</i>
P1-04	Booking cancellation OTP	Done	An email OTP, required after the policy is shown and before any state changes. A request without one is answered <code>otpRequired: true</code> rather than proceeding.
P1-05	Cancellation policy shown before confirmation	Done	<code>POST /user/cancel/quote</code> returns the exact refund for this booking under its own policy, and the page shows it before the guest confirms. The same policy computes the refund — snapshotted at booking, so a later change cannot alter what a guest was promised.
P1-06	Auto-accept must produce the intended booking outcome	Done	Accept records the agreed price per night, issues the server-side coupon, notifies both parties and routes the guest to checkout at that price.
P1-07	Engine and service must agree on boundaries	Done	One service owns the decision; the boundary cases (at the floor, one rupee below, at the ideal price) are unit-tested rather than described.
P1-08	Negotiated price must be authoritative	Done	The agreed price is a server-issued coupon pinned to the offer's dates. The client cannot name a price, and cannot move the dates the deal was struck for.
P1-09	Monthly stay flow complete	Done	Weekly and monthly rates are applied by the pricing engine for any stay long enough to qualify, on both normal and advance bookings, and flow through negotiation, booking, payment, cancellation and payout. Renewal is a new booking on the same path rather than a second mechanism.

ID	Requirement	Status	Detail
P1-10	Payout validated against available balance	Fixed	There was a comment saying the amount should be checked against the host's account, and directly under it, the insert. Nothing checked anything: a host with fifty rupees of earnings could request fifty lakh, and because pending requests are subtracted from the displayed balance, one bogus request drove their own earnings negative. One balance helper is now used by both the request and the display, and a second pending request is refused.
P1-11	Versioned API paths	Needs your specification — section 8	
P1-12	Rate limits reconciled with the specification	Partly done	The OTP numbers this document supplies are implemented. The credential limiter was running at 50 per 15 minutes , ten times its own documented value, because it had been raised "for demonstration" — that is fixed and verified live (<code>ratelimit-limit: 5</code>). For general and analytics limits we need your table; ours are 600/15 min general and 500/15 min admin.
P1-13	Upload limits and content validation	Done, except scanning	There were no size limits at all — the file said so. Now 8 MB for an image, 12 MB for a document, 25 files per request; explicit MIME allowlists; the declared type and the extension must agree; and every stored file is checked against its own magic number and deleted if the bytes contradict the name. Malware scanning is not done and is not claimed — it needs a scanning service and a decision about where it runs.
P1-14	SVG and unsafe active content	Done	The filter matched <code>mimetype.startsWith("image/")</code> , which is true of <code>image/svg+xml</code> — a script container served back from our own origin. SVG is off the allowlist and rejected by name, with a message that says why rather than "only image files are allowed".
P1-15	Payment UI must not declare success from the client callback	Done	Both Razorpay success handlers in the app <code>await</code> the backend verification and only show success when the server confirms; a failed verification shows a failure.

5. Section 4 — Android APK / frontend findings

ID	Requirement	Status
FE-01	No development API URL in the release artifact	Fixed — compiled out of release builds; the endpoint must be named at build time
FE-02	No test Razorpay key in the release artifact	Fixed — same mechanism; a production build needs your live key
FE-03	No localhost, developer-machine paths or debug configuration	Already clean — verified against the artifact, see section 2
FE-04	Real OTP/session/token lifecycle, not stub authentication	Already real — no stub or demo authentication anywhere in the app

ID	Requirement	Status
FE-05	Live property data	Already live — no seed or demo fixtures in customer flows
FE-06	Negotiation connected to the backend	Already connected — offer, host response, counter, decline, socket updates, expiry and the final price
FE-07	Booking uses the server response	Already true — server-calculated values only
FE-08	Payment through backend order and verification	Already true , and P1-15 above
FE-09	Cancellation/refund from the backend	Already true — policy, OTP and refund status all server-side
FE-10	Host onboarding real	Already true — KYC, status, property creation and payouts all on real APIs
FE-11	Logout invalidates the server session	Already true since the revocation store
FE-12	No no-op buttons	Already true — two TODO comments in the whole app, neither on a production control. The one control that deliberately does nothing is checkout in a release build carrying a test key, which refuses with a message rather than taking a payment that collects nothing
FE-13	No hardcoded stock images	Already true — a previous build filled empty listings with stock photographs, which advertised places that do not exist; there is no fallback image now, and a listing with no photographs looks like one
FE-14	Network and error states handled without false success	Already true — timeouts, offline and 4xx/5xx are surfaced; a 401 clears the session rather than leaving a signed-in shell
FE-15	Duplicate submission protection	Already true — checkout guards re-entry and disables its own controls while a submission is in flight
FE-16	Restart recovery from the server	Already true — booking and payment state is re-read from the server on return, never assumed from local state
FE-17	Secure client storage	Already true — the session token is held in <code>flutter_secure_storage</code> (Android Keystore), not in shared preferences
FE-18	TLS validation not bypassed	Already true — verified against the artifact and the manifest, see section 2

6. Section 5 — Backend / API findings

ID	Area	Status
BE-01	Secrets and configuration	Done — environment only; readiness reported without publishing the inventory
BE-02	Authentication on all protected APIs	Done — one verification path, with revocation and now the inactivity window

ID	Area	Status
BE-03	RBAC and ownership across guest, host, admin, finance	Done, and one gap closed — see P0-04
BE-04	Guest booking ownership	Done — every booking query carries the caller's id
BE-05	Host ownership	Done — every host query carries <code>property_host_id</code>
BE-06	Cancellation eligibility, OTP, refund	Done
BE-07	Payment secure, atomic, authorized, idempotent	Done — see P0-09
BE-08	Refund server-controlled and policy-derived	Done — from the policy snapshot
BE-09	Payout balance, eligibility, KYC, idempotency	Fixed — see P1-10
BE-10	Minimum/ideal hidden from the guest	Done — excluded for anyone but the owner; verified live on the public property payload
BE-11	Negotiation logging	Done — offer, decision, host action, outcome and price snapshot
BE-12	Upload storage, validation, scanning	Done except scanning — see P1-13
BE-13	API contract reconciled with the OpenAPI	Needs your specification — section 8
BE-14	Rate limits, stronger on sensitive endpoints	Fixed — see P1-12
BE-15	Restrict diagnostic endpoints	Fixed — this was live
BE-16	CORS and WebSocket security	Fixed — and this one was not visible from anywhere
BE-17	Privileged mutations traceable	Done — a durable admin audit ledger records actor, action, object, before/after and timestamp
BE-18	Financial mutations atomic	Done — booking, payment, cancellation and settlement each run in one transaction; two places where a transaction was silently dropped by a misplaced argument have been corrected

7. Section 6 — The critical end-to-end flow

Step	State
Guest opens the released APK against the approved environment	✅ build 30, endpoint named at build time

Step	State
OTP authentication and a valid session	✓
Search live properties, open a real one	✓
Nightly or monthly duration	✓
Offer without receiving minimum/ideal pricing	✓ verified on the live public payload
The locked negotiation decision applies	✓
Host receives and responds where escalation is required	✓
Accepted price becomes the booking price	✓ server-issued, date-pinned
Payment through the approved gateway	✓ sandbox until you supply the live key
Backend verifies and confirms exactly once	✓ row-locked, replay-safe
Booking appears in the right guest and host accounts	✓
Guest sees the applicable cancellation terms	✓
Cancellation requires OTP and produces the right refund	✓
Host reaches only their own properties, bookings and payouts	✓
Important operations auditable	✓
Restart or network interruption produces no false success or duplicate	✓

8. What we need from you

Four things, and none of them is work we can do on your behalf without guessing.

1. The live Razorpay key (P0-02). A production APK cannot be built without it. The build command and the verifier are ready:

```
./tool/build_release.ps1 -ApiBaseUrl https://... -RazorpayKey rzp_live_...
```

It refuses a test key unless `-AllowTestPayments` is passed, then reads the built APK back and fails if a test key, an unexpected endpoint, a developer path or a plain-http endpoint is in it. Build 30 is the QA candidate: it carries the sandbox key **because it was passed deliberately**, and the tool says so in its output rather than passing quietly.

2. The production endpoint (P0-01). There is one backend. `aaajaodev.onrender.com` is a Render *service* name and it is what `www.aaajoohomes.com` talks to — so the app was pointed at the right host with a name that reads like the wrong one. Pointing the app elsewhere would point it at a different database. Two options: keep it and accept the name, or put a custom domain (`api.aaajoohomes.com`) in front of the same service — a DNS record

and a Render setting, no code change, and then the APK contains a production-looking, production-stable name. **We recommend the second**, and it needs your DNS access. Either way the endpoint is now named explicitly at build time, so it can never be inherited by accident again.

3. The 60-minute access token and refresh model (P1-01). We have not done this, and we would rather say so than half-do it. It is one coordinated change across three codebases: the backend issues a short access token plus a revocable refresh token, and the website and the app each learn to refresh once on a 401 and retry. Done carelessly it either logs everyone out or loops. Our proposal: schedule it as its own change with its own test cycle immediately after final testing, rather than folding it into the release candidate now. The security it targets — a token that outlives its usefulness — is substantially covered today by server-side revocation and the 30-minute inactivity window that shipped with this build. **If you want it in this release instead, say so and we will do it, with the understanding that authentication needs re-testing end to end afterwards.**

4. The API specification for versioning, error codes and rate limits (P1-11, P1-12, BE-13). The document refers to an approved OpenAPI contract with versioned paths, a standard error-code set (`AUTH_001` , `PAYMENT_001/002` and the rest) and specified rate limits. We do not have the file. Adding a `/v1` alias is mechanical and backward-compatible, and adding a `code` field beside the existing message is additive and safe for both clients — but which condition maps to which code is not something to guess, because a contract that disagrees with your specification is worse than one not yet written. **Send the specification and all three become short pieces of work.**

9. Section 8 — Developer update

Required item	Response
Build/version	Android <code>1.0.0 (30)</code>
Backend release/version	<code>6ab9cbb</code> , deployed
Environment used for validation	The live backend and the live website; the APK scanned as a file
P0 findings closed	10 of 10. P0-01 and P0-02 are closed in the code; a <i>production</i> APK still needs your live payment key (section 8)
P1 findings closed	13 of 15. P1-01 proposed for its own cycle; P1-11 needs your specification
Database migrations included	One, from the same day's work: <code>20260906120000-guest-support-tickets</code> , already applied
Payment changes validated	Signature verification, replay, ownership, refresh and pending reconciliation — by code review and by the existing suite. Live gateway testing still uses the sandbox
Negotiation changes validated	Floor privacy verified on the live payload; auto-accept, escalation, expiry and concurrency covered by tests
Authentication/session changes validated	Revocation and the new inactivity window are unit-tested; the admin guard sweep is a test that fails the build
Android regression completed	Build 30 compiles clean and was scanned; we have not driven the new build through a signed-in journey on a device — see below

Required item	Response
Known remaining issues	The four items in section 8. Nothing else is open from this document
APK file/hash	Aajoo-Homes-v1.0.0-build30-qa.apk · 96.2 MB · SHA-256 8528db9d7b5cde1f6265c949a0ee13d8df6c933093292627b9cfc2a9a23bb4fa

One limit, stated plainly. We do not enter stored account passwords into forms, so the parts of the Android regression that need a signed-in session on a handset are yours to run rather than ours. Everything that could be verified without signing in — the artifact's contents, the build configuration, the backend behaviour, the unit suite — has been.

10. Release position

● **The eight defects are fixed and deployed. Two of them were live and neither was visible from any screen.**

The two that matter most, said plainly:

- **An admin endpoint that took no authentication.** `/admin/properties/load` answered on production and inserts property rows from a spreadsheet. It is guarded now, and a sweep test fails the build if any `/admin` route ever appears without a guard again.
- **Sockets that were anonymous while looking authenticated.** Any socket that could connect could send a chat or negotiation message as any user id. The app was sending its token in a header the server never read.

Everything else in this document was either already implemented — and is now stated with the evidence rather than the assertion — or is one of the four things in section 8 that need you.

Android	1.0.0 (30) — QA candidate, sandbox gateway
Backend	6ab9cbb — 62/62 test files
Website	b8f3818
New tooling	tool/build_release.ps1, tool/verify_release_apk.py
New tests	tests/apkFindings.test.js (21 assertions), tests/socketAuth.test.js (6, driving a real handshake)
Live verification after deploy	/admin/properties/load → 401 · /admin/properties/search → 401 · /health/push → {"pushReady":true} and nothing else · an unauthenticated socket → Authentication required
APK evidence	A release build with no payment key passed contains no <code>rzp_test</code> string — scanned under production rules, clean. The deliverable QA build carries the sandbox key because it was passed deliberately
APK SHA-256	8528db9d7b5cde1f6265c949a0ee13d8df6c933093292627b9cfc2a9a23bb4fa